

Toán học của Batch Normalization

Lớp BatchNormalization chuẩn hóa từng lô (Batch) trong lúc Training:

- 1 Tính Kỳ vọng (μ) của Batch: $\mu = \frac{1}{m} \sum_{i=1}^m x_i$
- 2 Tính Phương sai (σ^2) của Batch: $\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$
- 3 Chuẩn hóa gốc: $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$
- 4 **Dịch chuyển và Tỷ lệ (Scale & Shift):** $y_i = \gamma \hat{x}_i + \beta$

Trong đó, γ và β là 2 tham số **được học** qua Gradient Descent. Nó cho phép mô hình *hủy bỏ* tác dụng của BN và khôi phục phân phối gốc nếu việc chuẩn hóa làm mất đi thông tin quý giá.

Quá trình Suy luận (Inference Mode)

- Khi Đưa vào sử dụng (Inference/Predict), ta thường dự đoán trên từng ảnh đơn lẻ (Batch Size = 1). Không thể tính μ và σ^2 của 1 ảnh được!
- Giải pháp: Trong suốt quá trình Training, lớp BN âm thầm cập nhật một **Trung bình động lũy thừa (Exponential Moving Average)** của các μ và σ^2 từ tất cả các batches.
- Khi Inference, nó dùng hằng số Moving Average này để chuẩn hóa thay vì tính toán lại.
- *Lưu ý khi Transfer Learning*: Luôn đóng băng (Freeze) lớp BN bằng 'trainable=False' để tránh làm hỏng Moving Average đã được học rất tốt từ ImageNet.

Cách lập trình Batch Normalization chuẩn xác

Vì BN tự động chuẩn hóa Mean về 0, tham số bias của tầng Convolution trước đó trở nên vô dụng (bị hấp thụ bởi β). Ta nên tắt nó bằng `use_bias=False` để tiết kiệm RAM.

Cách KHÔNG NÊN dùng (Activation bị đặt sai chỗ):

```
1 x = layers.Conv2D(32, 3, activation="relu")(x)
2 x = layers.BatchNormalization()(x)
```

Cách SỬ DỤNG CHUẨN (BN phải đứng TRƯỚC Hàm kích hoạt):

```
1 # Note the lack of bias and activation here
2 x = layers.Conv2D(32, 3, use_bias=False)(x)
3 x = layers.BatchNormalization()(x)
4 # We place the activation AFTER the BatchNormalization layer
5 x = layers.Activation("relu")(x)
```

Giới hạn của Tích chập Truyền thống (Conv2D)

- Lớp Conv2D thực hiện đồng thời 2 việc trong 1 bước tính toán duy nhất:
 - ① Học các mẫu Không gian 2D (Spatial Pattern - VD: đường viền).
 - ② Tổ hợp thông tin chéo từ nhiều Kênh (Cross-channel Mixing - VD: Kết hợp kênh Đỏ và Xanh).
- Chi phí tính toán cực kỳ tốn kém, số tham số: $K^2 \times C_{in} \times C_{out}$.
- **Giả thuyết mới:** Đặc trưng Không gian và Đặc trưng Kênh có tính độc lập cao. Tại sao không tách chúng ra?

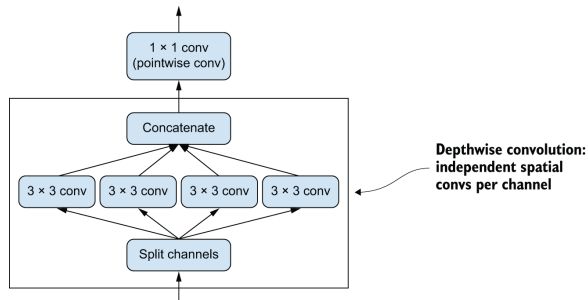
Tích chập Tách rời (Depthwise Separable Convolution)

Tách phép tính thành 2 pha độc lập
(SeparableConv2D):

- 1 **Depthwise Conv:** Quét không gian trên từng Kênh một cách độc lập (Không trộn lẫn).
- 2 **Pointwise Conv (1×1):** Nhào trộn thông tin giữa các Kênh tại từng điểm ảnh.

Số lượng tham số giảm mạnh:

$$K^2 \times C_{in} + C_{in} \times C_{out}.$$



Hình 9.4: Tích chập Tách rời chiều sâu

- Giả sử ta dùng Kernel 3×3 , đầu vào có 64 Channels, đầu ra có 64 Channels.
- **Tích chập chuẩn (Conv2D):**
 - Số tham số $= 3^2 \times 64 \times 64 = 36,864$ tham số.
- **Tích chập Tách rời (SeparableConv2D):**
 - Số tham số $= (3^2 \times 64) + (64 \times 64) = 576 + 4,096 = 4,672$ tham số.
- **Kết luận:** Tiết kiệm gần 90% bộ nhớ và năng lực tính toán. Mô hình hội tụ nhanh hơn, chống Overfitting tốt hơn do ít tham số dư thừa.

Sự co-evolve (Tiến hóa đồng thời) của Phần cứng và Phần mềm

- Dù tiết kiệm 90% tham số, nhưng trên GPU thực tế, SeparableConv2D chỉ nhanh **tương đương** Conv2D thông thường. Tại sao?
- Bởi vì **NVIDIA cuDNN** đã tối ưu vi mô (micro-optimized) kernel Conv2D tiêu chuẩn bằng hợp ngữ (Assembly) qua hàng thập kỷ cho hàng Exaflops tính toán.
- Các thuật toán mới dù ưu việt về toán học nhưng không được phần cứng ưu tiên thiết kế → Sẽ chạy rất chậm.
- Đây là rào cản khổng lồ (Sự tiến hóa đồng thời của Hardware - Software - Algorithm) ngăn chặn các đột phá AI đi chệch khỏi lối mòn của Backpropagation.

Lắp ráp mô hình Mini-Xception (Phần 1)

Chương trình sau ứng dụng cả 3 bảo bối: **SeparableConv**, **BatchNormalization**, và **Residual Connections**.

```
1 inputs = keras.Input(shape=(180, 180, 3))
2 x = layers.Rescaling(1.0 / 255)(inputs)
3
4 # First layer must be regular Conv2D because RGB channels
5 # are highly correlated in natural images. Separable assumption fails here!
6 x = layers.Conv2D(filters=32, kernel_size=5, use_bias=False)(x)
7
8 for size in [32, 64, 128, 256, 512]:
9     residual = x
10
11     x = layers.BatchNormalization()(x)
12     x = layers.Activation("relu")(x)
13     x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
```

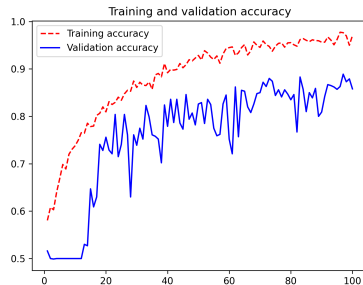
Lắp ráp mô hình Mini-Xception (Phần 2)

Kết thúc vòng lặp, ta gắn khối Classifier (Toàn cục). Kích thước mạng cực nhẹ, chỉ tốn 721,857 tham số so với 1.5 triệu tham số của CNN cũ!

```
1 x = layers.BatchNormalization()(x)
2 x = layers.Activation("relu")(x)
3 x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
4 x = layers.MaxPooling2D(3, strides=2, padding="same")(x)
5
6 # Project residual block
7 residual = layers.Conv2D(size, 1, strides=2, padding="same", use_bias=False)(
8   residual)
9 x = layers.add([x, residual])
10
11 x = layers.GlobalAveragePooling2D()(x)
12 x = layers.Dropout(0.5)(x)
13 outputs = layers.Dense(1, activation="sigmoid")(x)
14 model = keras.Model(inputs=inputs, outputs=outputs)
```

Đánh giá Độ chính xác (Accuracy)

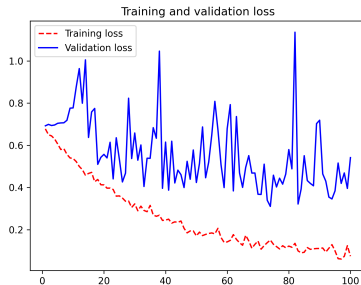
- Nhờ cấu trúc siêu ưu việt, mô hình tự thiết kế của chúng ta đạt **Accuracy lên đến 90.8%** trên tập Chó/Mèo.
- Mô hình Basic CNN ở Chương 8 (Không dùng Transfer Learning) chỉ đạt được mốc 83.9%.
- Thiết kế kiến trúc chuẩn mực (Architecture Best Practices) mang lại **Bước nhảy vọt** về hiệu năng!



Hình 9.5: Accuracy của Mini-Xception

Đánh giá Mất mát (Loss)

- Đường Validation Loss đi xuống ổn định và mượt mà hơn.
- Sự hiện diện của BatchNormalization ở mọi ngõ ngách giúp thuật toán hội tụ cực nhanh.
- Dạng kiến trúc lặp (Xception) hiện là tiêu chuẩn công nghiệp (Industry Standard) trên Keras.



Hình 9.6: Loss của Mini-Xception

Giao thời của Công nghệ (Beyond Convolution)

- Mạng Tích chập (ConvNets) đã độc tôn thống trị lĩnh vực Thị giác Máy tính suốt từ thập niên 2010.
- Gần đây, một Kiến trúc hoàn toàn mới đang đe dọa thay thế ngôi vương này: **Vision Transformers (ViTs)**.
- Transformers vốn dĩ được sinh ra vào năm 2017 dành riêng cho Xử lý Ngôn ngữ Tự nhiên (NLP - để dịch thuật văn bản).
- Ý tưởng táo bạo: Chuyện gì sẽ xảy ra nếu ta coi một bức ảnh giống như là "Một câu văn", và các bộ phận của ảnh là "Các từ vựng"?

- Mạng ViT hoàn toàn **bác bỏ** toán tử Conv2D để quét ảnh.
- Nó chặt bức ảnh gốc thành một lưới các mảnh ghép (Patches) có kích thước cố định, ví dụ 16×16 pixel.
- Trải phẳng mỗi mảnh ghép thành một Vector 1 chiều (Biến nó thành một "Từ vựng" - Token).
- Bơm toàn bộ chuỗi Vector vào kiến trúc Transformer để dùng cơ chế **Self-Attention** tìm kiếm mối tương quan.

Sức mạnh của Self-Attention

- Mạng ConvNet gặp khó khăn trong việc kết nối các chi tiết ở xa nhau trên ảnh do kích thước Kernel (3×3) quá nhỏ bé. Phải xếp lớp rất sâu mới nhìn được toàn cảnh.
- Mạng ViT với cơ chế Self-Attention có khả năng so sánh **mọi Patch với mọi Patch khác** cùng một lúc.
- Điều này tạo ra khả năng liên kết cự ly xa (Long-range relationships) tuyệt vời. Nó hiểu ngay lập tức quả bóng ở góc trái có liên quan mật thiết đến con chó đang chạy ở góc phải bức ảnh.

Hạn chế của ViTs so với ConvNets

Tuy rất mạnh, ViTs có những điểm yếu chí mạng đối với các dự án thực tế:

- **Cần dữ liệu khổng lồ:** ConvNets có lợi thế "Kiến thức ưu tiên 2D" (Biết rõ các pixel cạnh nhau thì liên quan đến nhau). ViTs là một tờ giấy trắng, nó phải tự học lại định luật vật lý về không gian → Phải đào tạo trên lượng dữ liệu khổng lồ (Lớn hơn cả ImageNet).
- **Mô hình quá đồ sộ:** Mạng ViT cực nặng, đòi hỏi hạ tầng Siêu máy tính (Clusters of TPUs/GPUs) khổng lồ để huấn luyện.
- Do đó, với các bài toán doanh nghiệp thông thường và dữ liệu có giới hạn, **ConvNets vẫn là sự lựa chọn hoàn hảo nhất.**